Wydział
Budowy Maszyn i Informatyki

Uniwersytet
Bielsko-Bialski

Technologie Blockchain

SPRAWOZDANIE

Laboratorium nr 2

Temat: Transakcje i wydobywanie bloków w prototypowym łańcuchu bloków

KOŁODZIEJCZYK KAROLINA
Kierunek: Cyberbezpieczeństwo II stopnia
Semestr: I zaoczny
Grupa: 2a

Link: https://github.com/ryzykotolerowalne/TB_lab_sem1/blob/main/main2.py

Cel ćwiczenia:

Rozszerzenie prototypu blockchaina o pulę transakcji oczekujących i mechanizm wydobywania bloków Proof of Work oraz zbadanie wpływu trudności na koszt obliczeniowy i bezpieczeństwo łańcucha.

Opis ćwiczenia:

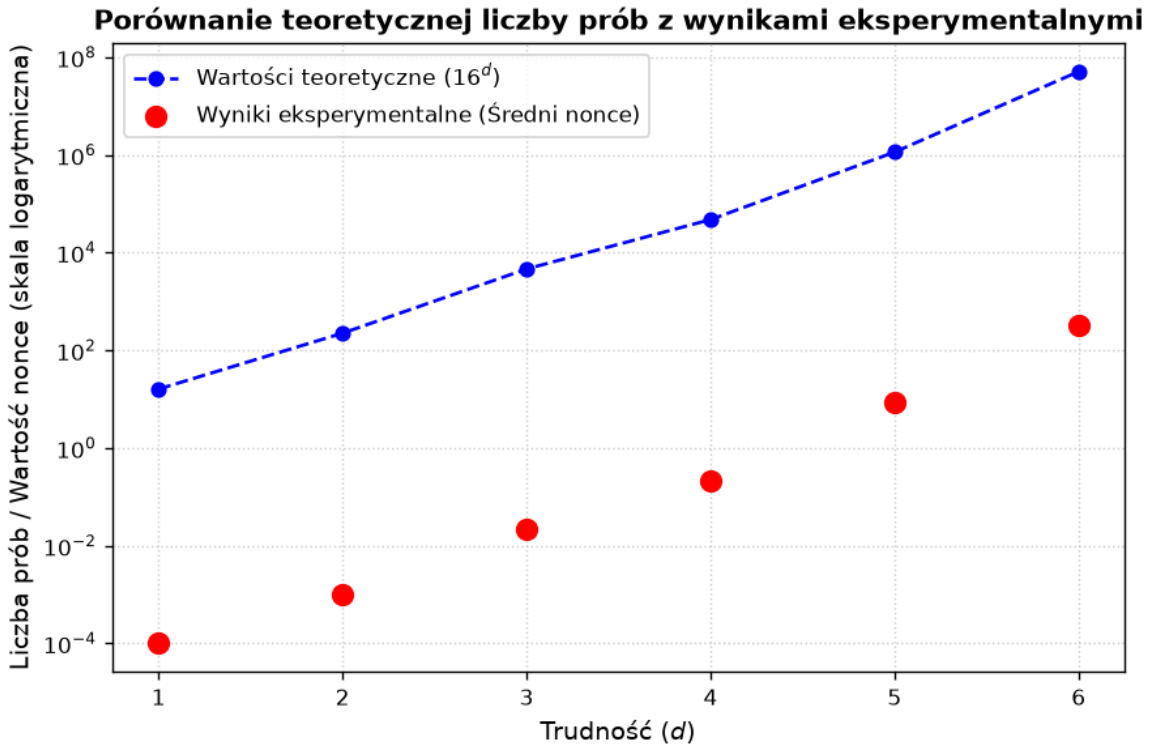
Przeanalizowano uważnie treść instrukcji laboratorium wraz z przykładami 1-4. Na podstawie poprzedniego zadania została rozbudowana klasa Blockchain z ćwiczenia nr 1. Dodano kilka transakcji, wydobyte zostały dodatkowe bloki i wyświetlony został łańcuch cały łańcuch. Dowiedziono, że skrót każdego bloku zaczyna się od wymaganej liczby zer.

```
C:\Users\karol\OneDrive\Desktop\TB\zad2_17102026\.venv\Scripts\python.exe C:\Users\karol\OneDrive\Desktop\TB\zad2_17102026\main.py
2.1: WYDOBYCIE BLOKÓW TESTOWYCH
Blok 1 | Nonce: 0 | Hash: 147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172
Blok 2 | Nonce: 52186 | Hash: 000073ef1eceef857ae778bb770784893f15f0b025812c1af15ca9deb82d51da
Blok 3 | Nonce: 22719 | Hash: 0000568cfa1c0fb0ac55d639c949a3cd9a6c3bd7869f836b14bab4722670816f
Blok 4 | Nonce: 77029 | Hash: 000050b0a51768c6ccf803d6fdc77e9101ef88bbef1eb7d99c391002326c20c8
----------------------------------------------------------------------
```

Dla trudności od 1 do 6 wydobyto po 5 bloków i zmierzono średni czas wydobycia oraz średnią wartość nonce. Poniżej tabela dla poszczególnych wartości.

Trudność	Teoretyczna liczba prób 16^d	Średnia wartość nonce	Średni czas [s]
1	16	0.0001	208045
2	223	0.0010	223554
3	4596	0.0212	216313
4	47205	0.2109	223792
5	1136743	8.3638	135912
6	50486879	328.3615	153753

Obliczono wydajność komputera w skrótach na sekundę. Wyniki zostały przedstawione na wykresie i porównane z wartościami teoretycznymi.



Na podstawie aktualnej mocy obliczeniowej sieci Bitcoin (np. https://mempool.space/mining) oszacowano, że temu komputerowi wydobycie jednego bloku Bitcoin zajęłoby x lat.

Wydobyto łańcuch złożony z 5 bloków (trudność 4), a następnie zmieniono kwotę transakcji w bloku nr 2. Pokazano, że valid_chain() zwraca False.

Napisano funkcję symulującą atakującego, który po modyfikacji ponownie wydobywa blok nr 2 i wszystkie kolejne bloki. Zmierzono czas ataku dla trudności 4 i 5.

```
2.3: ATAK NA INTEGRALNOŚĆ ŁAŃCUCHA
Stan przed atakiem: valid_chain() -> False
Po manipulacji kwotą: valid_chain() -> False
Czas potrzebny na sfałszowanie całego łańcucha (d=4): 1.1722 s
Czas potrzebny na sfałszowanie całego łańcucha (d=5): 18.2676 s
----------------------------------------------------------------------
```

Zależność czasu ataku od trudności oraz liczby dobudowanych bloków

Czas potrzebny na przeprowadzenie udanego ataku (czyli retroaktywnego sfałszowania historii transakcji w taki sposób, aby sieć uznała ją za poprawną) zależy od dwóch czynników:

Od trudności (\(d\)): Czas ten rośnie w sposób wykładniczy (geometryczny). Ponieważ funkcja skrótu SHA-256 generuje wyniki w systemie szesnastkowym, każde podniesienie parametru trudności o 1 wymusza znalezienie dodatkowego wiodącego zera. Zawęża to przestrzeń

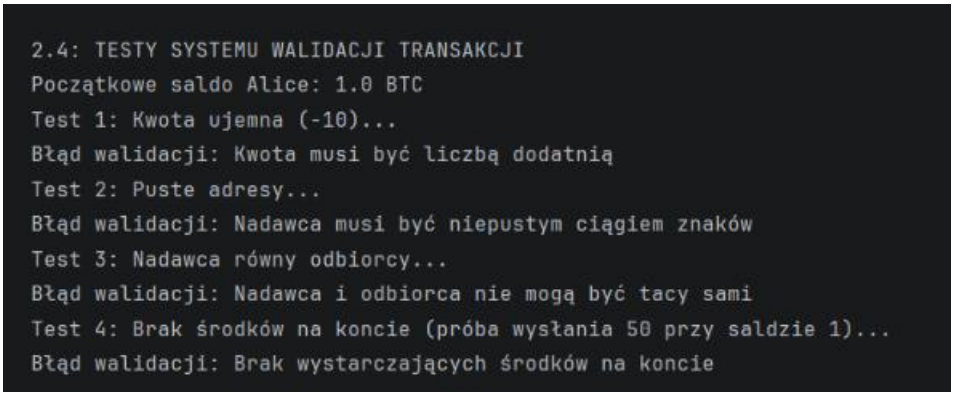
pasujących hashy szesnastkowych średnio 16-krotnie na każdy blok. Haker musi wykonać o tyle samo więcej operacji matematycznych, co drastycznie wydłuża czas trwania ataku wraz ze wzrostem \(d\).

Od liczby dobudowanych bloków: Wpływ ten ma charakter liniowo-skumulowany. Zmiana danych w jakimś bloku historycznym całkowicie unieważnia jego hash, co zrywa kryptograficzne powiązanie z blokiem następnym. Atakujący nie może poprzestać na sfałszowaniu jednego bloku – aby stworzyć spójną strukturę, musi od nowa przeprowadzić proces Proof of Work (kopanie do skutku) dla zmodyfikowanego bloku oraz dla każdego kolejnego bloku, który został nad nim nadbudowany aż do czoła łańcucha. Im więcej bloków dobudowano nad transakcją, tym więcej niezależnych i kosztownych procesów wydobywczych haker musi wykonać, aby dogonić sieć.

Otrzymanie 6 potwierdzeń oznacza, że transakcja znajduje się w bloku, nad którym sieć pomyślnie dobudowała już 5 kolejnych bloków. W architekturze Bitcoina obowiązuje zasada konsensusu mówiąca, że węzły zawsze uznają za prawdziwą tę wersję historii, która reprezentuje najdłuższy i najbardziej wymagający obliczeniowo łańcuch (ang. longest chain rule).

Aby cofnąć lub podmienić transakcję mającą 6 potwierdzeń, atakujący musiałby w pojedynkę stworzyć alternatywną gałąź łańcucha o długości co najmniej 6 bloków i rozgłosić ją w sieci szybciej, niż globalna moc wszystkich pozostałych górników dobuduje kolejne bloki do głównego łańcucha. Z matematycznego punktu widzenia (analiza losowych procesów stochastycznych opisana w dokumencie Satoshi Nakamoto), prawdopodobieństwo sukcesu takiego ataku przez podmiot dysponujący nawet bardzo dużą (ale mniejszą niż 50%) mocą obliczeniową spada asymptotycznie do zera. Koszt energii elektrycznej i sprzętu potrzebnego do przeprowadzenia takiego ataku drastycznie przewyższa potencjalne korzyści finansowe z oszustwa, co czyni transakcję z 6 potwierdzeniami absolutnie bezpieczną i ekonomicznie nieodwracalną.

Dodano w metodzie new_transaction() walidację danych: kwota musi być liczbą dodatnią, nadawca i odbiorca muszą być niepustymi ciągami znaków i nie mogą być sobie równi. Niepoprawne transakcje powinny zgłaszać wyjątek ValueError.



Metoda get_balance() działa w sposób lokalny i sekwencyjny. Przeszukuje ona wyłącznie pamięć operacyjną jednego, konkretnego komputera (węzła) – analizuje zatwierdzony lokalnie łańcuch bloków (self.chain) oraz lokalną pulę transakcji oczekujących (self.current_transactions).

Dzięki temu węzeł skutecznie zapobiega oszustwu, jeśli użytkownik wyśle do niego dwie sprzeczne transakcje (np. mając 1 BTC, spróbuje w tym samym momencie wysłać 1 BTC do użytkownika X oraz 1 BTC do użytkownika Y). Druga transakcja w kolejności zostanie odrzucona, ponieważ funkcja get_balance() natychmiast wykaże, że po uwzględnieniu pierwszej transakcji saldo nadawcy spadło do zera.

Mechanizm ten jest bezradny, gdy ten sam oszust wykorzysta strukturę sieci rozproszonej i wyśle transakcję A do Węzła nr 1 (w Europie), a transakcję B (tymi samymi środkami) w tym samym ułamku sekundy do Węzła nr 2 (w Azji). Z perspektywy Węzła nr 1 transakcja A jest w pełni legalna i ma pokrycie w saldzie. Z perspektywy Węzła nr 2 transakcja B również jest poprawna. Bez dodatkowej komunikacji oba węzły zaakceptują te sprzeczne operacje i dopiszą je do swoich własnych, lokalnych wersji historii, doprowadzając do rozbieżności w sieci.

Aby zapobiec permanentnemu rozwidleniu sieci (ang. fork) i globalnie wyeliminować problem podwójnego wydawania, sieć rozproszona musi wdrożyć dwa fundamentalne komponenty architektoniczne:

Globalny protokół konsensusu (np. reguła Proof of Work i najdłuższego łańcucha)

Sieć typu Peer-to-Peer (P2P) z mechanizmem rozgłaszania (ang. Gossip Protocol):

Pytania kontrolne i odpowiedzi:

1. Na czym polega asymetria kosztu obliczeniowego w mechanizmie Proof of Work?

Znalezienie rozwiązania (wykopanie bloku) wymaga od procesora ogromnego czasu i milionów prób przeliczenia nonce, podczas gdy jego weryfikacja przez inne komputery w sieci jest natychmiastowa i wymaga zaledwie jednego kliknięcia (jednego hashowania).

2. Ile razy średnio wzrasta liczba prób po zwiększeniu trudności o jedno zero szesnastkowe?

Liczba prób wzrasta średnio **16-krotnie**. Wynika to z faktu, że system szesnastkowy (heksadecymalny) używa 16 znaków, więc każde kolejne wymagane zero na początku hashu zawęża pulę prawidłowych wyników dokładnie 16 razy.

3. Czym jest transakcja coinbase i dlaczego jest umieszczana w bloku jako pierwsza?

Jest to nagroda finansowa dla górnika za wykopanie bloku, tworząca nowe monety „z niczego" (brak nadawcy). Jest umieszczana jako pierwsza, aby sieć mogła ją błyskawicznie zidentyfikować i sprawdzić poprawność jej naliczenia przed weryfikacją reszty przelewów.

4. Dlaczego koszt modyfikacji transakcji rośnie wraz z liczbą potwierdzeń?

Zmiana danych w bloku niszczy jego hash, co zrywa połączenie z kolejnymi klockami. Haker musi wykonać proces Proof of Work od nowa dla sfałszowanego bloku oraz dla każdego bloku, który został nad nim nadbudowany (liczba potwierdzeń) – praca i koszt prądu rosną więc liniowo z każdym kolejnym blokiem.

5. Na czym polega podwójne wydawanie (double spending)?

To próba oszukania systemu i wydania tych samych pieniędzy cyfrowych dwukrotnie w dwóch różnych transakcjach jednocześnie (co w tradycyjnej bankowości uniemożliwia centralny bank, a w blockchainie blokuje mechanizm konsensusu).